

● Telecom Networks

Support AllLama

Manual Corporativo y Documentación Técnica de Operación

Sistema:	Asistente de IT Support Híbrido (Chat & WhatsApp Bot)
Organización:	Telecom Networks
Versión:	1.0.0 (Producción)
Fecha:	Julio 2026
Autoría:	IT Support Operations & Development Team

1. Introducción del Proyecto

1.1 Visión General

Support AILlama es la plataforma corporativa unificada de asistencia técnica y administración de TI diseñada para **Telecom Networks**. Integra en un único ecosistema web de alto rendimiento una interfaz de chat inteligente para los empleados y un panel administrativo para los ingenieros de soporte, complementado por un bot automático que interactúa con los usuarios a través de WhatsApp Web.

1.2 Objetivo Principal

Automatizar el primer nivel de asistencia técnica para incidentes cotidianos mediante inteligencia artificial local y en la nube. De esta forma, se resuelven problemas repetitivos de forma inmediata sin intervención humana directa, reduciendo los tiempos de inactividad de los agentes y permitiendo al equipo de TI concentrar sus recursos en incidentes de alta complejidad.

1.3 Problema que Resuelve

- **Saturación de la mesa de ayuda:** Soporte de primer nivel inundado por problemas recurrentes como configuraciones de audio, caídas de aplicaciones o fallas de VPN.
- **Lentitud en la resolución:** Retraso en la resolución de incidentes sencillos debido al flujo manual de asignación de tickets.
- **Falta de aislamiento y seguridad:** Evita que usuarios finales manipulen configuraciones del servidor o accedan a datos del ecosistema de TI, aplicando principios de *Zero Trust*.

1.4 Usuarios Destinatarios

Perfil	Descripción	Acceso de Interfaz
Empleados / Agentes	Personal de Telecom Networks (en sitio o WFH) con problemas de TI.	Chat Widget asistido por IA (solo lectura del log del ticket).
Administradores de TI	Personal autorizado de soporte técnico e infraestructura.	Acceso al panel administrativo con control total y monitoreo de APIs.

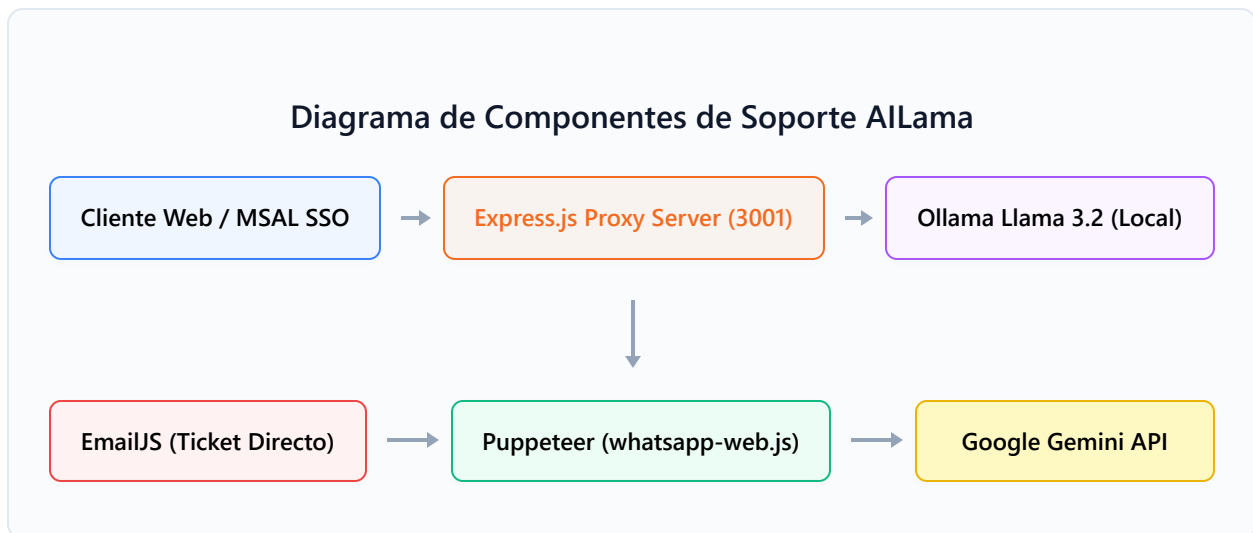
1.5 Flujo General de Funcionamiento

El sistema se inicia con una barrera de autenticación que exige el inicio de sesión corporativo en Microsoft. Una vez validadas las credenciales, el sistema enruta automáticamente al usuario según su rol de correo. El chat guía al usuario a través de un máximo de tres soluciones propuestas por la IA local (Ollama - Llama 3.2). Si la falla persiste, el sistema recopila su correo electrónico de contacto y genera automáticamente un ticket de soporte enviando el historial detallado de la conversación a través de EmailJS y registrando el ticket en la base de datos local para su visualización en el Panel de Administración.

2. Arquitectura y Estructura del Sistema

2.1 Arquitectura General

La aplicación está diseñada sobre una arquitectura desacoplada híbrida. El frontend es una Single Page Application (SPA) construida en JavaScript vanilla que se comunica de manera asíncrona con el servidor local Node.js/Express.js (que actúa como pasarela/proxy seguro). A su vez, se realizan integraciones directas con Microsoft Entra ID para la autenticación y EmailJS para la distribución de correos del Helpdesk, reduciendo la exposición de las credenciales de SMTP.



2.2 Tecnologías Utilizadas

- **Frontend:** HTML5 estructurado semánticamente, CSS3 con variables CSS para el theming (Light/Dark mode) y diseño basado en *Glassmorphism*, JavaScript asíncrono puro (ES6), MSAL.js v2 para SSO con Microsoft, y EmailJS SDK.
- **Backend:** Node.js con Express.js (v5.2.1), administrado en el servidor Linux mediante PM2 (Process Manager).
- **Persistencia de Datos:** Archivo JSON estructurado `support\_tickets.json` para registrar los incidentes y `whatsapp\_exclusions.json` para definir exclusiones de números telefónicos y grupos de WhatsApp.

- **Motores de Inteligencia Artificial:** Instancia local de Ollama ejecutando `llama3.2:latest` para generar pasos sencillos de resolución de problemas, e integración con Google Gemini v1.5 Flash para tareas de resúmenes de tickets e informes ejecutivos de gestión mensual y semanal.
- **Integración WhatsApp:** Módulo `whatsapp-web.js` ejecutado en Puppeteer headless en Linux para automatizar respuestas en chats y notificar de reportes mensuales.

2. Estructura y Dependencias del Proyecto

2.3 Estructura de Carpetas

```
support_assistant/  
├── .env                # Configuración de variables de entorno y API keys  
├── .gitignore          # Exclusiones de archivos para control de versiones Git  
├── package.json        # Definición del proyecto Node.js y dependencias  
├── server.js           # Servidor backend Express y control del bot de WhatsApp  
├── index.html          # Página de entrada, Login MSAL, Gateway y chat del cliente  
├── styles.css          # Estilos CSS generales y theming para index.html  
├── script.js           # Controlador del frontend, lógica del chat e IA  
├── admin.html          # HTML del panel administrativo web (integrado en SPA)  
├── admin.css           # Estilos CSS específicos para la vista de administración  
├── admin.js            # Lógica de administración, monitoreo de APIs y tickets  
├── deploy.js           # Script de automatización de despliegue SSH/SFTP  
├── remote_check.js     # Script para pruebas de diagnóstico SSH remotas  
├── support_tickets.json # Archivo JSON de base de datos local para tickets  
├── whatsapp_exclusions.json # Archivo JSON para exclusión de números y grupos  
├── .wwbjs_auth/        # Carpeta generada automáticamente por WhatsApp Web  
└── .wwbjs_cache/       # Caché del navegador Chromium de Puppeteer
```

2.4 Dependencias de Terceros (package.json)

Dependencia	Versión	Función en el Sistema
@google/generative-ai	^0.24.1	Interacción directa con la API en la nube de Google Gemini.
express	^5.2.1	Servidor web que expone las APIs locales de control y sirve archivos.
whatsapp-web.js	^1.34.6	Control de sesión de WhatsApp Web y envío de mensajes salientes.
node-cron	^4.2.1	Programación horaria para reportes semanales por email y mensuales en WA.
nodemailer	^8.0.5	Envío de correos semanales con reporte ejecutivo desde el servidor.
ssh2-sftp-client	^12.1.0	Transferencia automatizada de archivos SFTP en el script de despliegue.
dotenv	^17.3.1	Carga de variables de entorno desde el archivo .env .

3. Desarrollo e Implementación

3.1 Lógica Híbrida de Inteligencia Artificial

El sistema combina de manera óptima modelos de IA de red local y en la nube:

- **Ollama (Llama 3.2):** Utilizado para el diálogo interactivo con el usuario (soporte paso a paso). Las llamadas se procesan en la red local de la empresa, garantizando latencias reducidas y cero transferencia de datos internos de la organización a redes externas.
- **Google Gemini 1.5 Flash:** Usado en la generación de resúmenes de tickets e informes ejecutivos de gestión mensual y semanal. Su capacidad para analizar grandes contextos textuales permite condensar docenas de logs de chats técnicos en directrices breves para la gerencia.

3.2 Restricciones de Nivel de Usuario en Prompting

Para evitar daños accidentales en las estaciones de trabajo de los agentes, la IA local de soporte está fuertemente limitada en sus prompts de sistema. El servidor aplica reglas estrictas a nivel de backend y frontend:

```
promptContext = `You are AILlama, a professional IT Support Assistant. User issue: "${userIss  
Provide exactly ONE simple, basic user-level troubleshooting step they can easily try.  
IMPORTANT: Avoid advanced technical steps like Safe Mode, BIOS, command line, or network conf  
Keep it brief and easy to understand for non-technical users.`;
```

3.3 Integración de Autenticación MSAL SSO

Para cumplir con las normas de seguridad de Telecom Networks, se implementó el protocolo OAuth 2.0 y OpenID Connect a través de la biblioteca MSAL (Microsoft Authentication Library) en el cliente. La aplicación está registrada en el directorio corporativo (Azure AD) con la siguiente configuración fija:

- **Client ID:** 10f60053-8a04-455c-8aa7-9b2235fb1051

- **Tenant ID:** `f209f7d9-4d32-41fb-b0b6-a23a382312f0`
- **Redirect URI:** `https://supportia.telecomnetworks.org/`

Esta autenticación provee un token de identidad cifrado (JWT) directo. La aplicación extrae el nombre y correo del inquilino oficial de Windows. Si el inicio de sesión es exitoso, la función `showRoleSelectionOrApp()` evalúa el dominio y nombre de la cuenta para validar privilegios de administrador de TI.

3. Desarrollo e Implementación (Flujo Técnico)

3.4 Distribución de Reportes Automatizados

El backend cuenta con dos tareas programadas mediante la librería `node-cron` :

1. **Reporte Semanal (Email):** Se ejecuta cada lunes a las 08:00 AM (`0 8 * * 1`). Consulta la base de datos de tickets registrados en los últimos 7 días, genera un resumen analítico con la API de Google Gemini en español y lo envía por correo electrónico a la terna administrativa corporativa (`rurena@tnoutsourcing.com` , `mbertoizzo@tnoutsourcing.com` , `ireyes@tnoutsourcing.com`).
2. **Reporte Mensual (WhatsApp):** Se ejecuta el primer día de cada mes a las 10:00 AM (`0 10 1 * *`). Filtra los tickets correspondientes al mes anterior, utiliza Ollama local para resumir las tendencias principales de soporte en 3 líneas en español y lo envía por WhatsApp a los números de los administradores técnicos registrados.

3.5 Variables de Entorno del Ecosistema (.env)

El backend de Express.js requiere que se defina un archivo `.env` en la raíz del directorio con las siguientes credenciales e IPs:

```
# --- Google API Keys ---
GEMINI_API_KEY=AIZA... # Key de acceso a Google Gemini API

# --- Ollama AI Configuration ---
OLLAMA_HOST=http://192.168.50.233:11434 # URL/IP local de servidor de Ollama

# --- SMTP Server Configuration (Nodemailer) ---
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=soporte@tnoutsourcing.com
SMTP_PASS=tu_clave_de_aplicacion_aqui
```

3.6 Envío Asíncrono de Tickets a Mesa de Ayuda (EmailJS)

Cuando un agente del chat web confirma que las 3 opciones iniciales de la IA fallaron, la aplicación recopila su correo electrónico de contacto y envía una solicitud HTTP asíncrona directamente a los servidores de EmailJS. Se utiliza la plantilla `template_t961pe6` bajo el servicio `service_9q5tkyt` y el User ID público de inicialización `Qob3T9TZxrEeMUz3G`. Esto garantiza el envío inmediato del ticket a la cola de soporte sin exponer contraseñas de correos compartidos en el código del cliente.

4. Manual de Usuario Final

4.1 Acceso al Sistema

Para ingresar al portal de asistencia técnica de Telecom Networks, abra el navegador web (se recomienda Google Chrome, Microsoft Edge o Firefox) e ingrese a la dirección URL oficial del sistema:

<https://supportia.telecomnetworks.org/>

4.2 Pantalla de Inicio de Sesión (Microsoft SSO)

Al acceder, el sistema desplegará un formulario corporativo seguro. Haga clic en el botón "**Sign in with Microsoft**". Se abrirá la pasarela oficial de inicio de sesión de Azure AD. Ingrese sus credenciales de correo institucional (ejemplo: `usuario@tnoutsourcing.com`) y la clave habitual de su estación de trabajo Windows. Al autenticarse con éxito, retornará automáticamente a la aplicación.

4.3 Perfilamiento de Conexión del Agente

Antes de habilitar el chat de soporte, se mostrará una ventana con un formulario obligatorio para identificar la ubicación física y logística de la estación de trabajo:

1. **Work Location:** Seleccione si se encuentra físicamente en las oficinas de la empresa (**Work On Site**) o si está en teletrabajo (**Work From Home**).
2. **Agent Name:** Este campo se pre-rellena automáticamente con el nombre de su cuenta de Microsoft corporativa.
3. **Project Name:** Escriba el nombre del proyecto o departamento al que está asignado.
4. **Box Number:** Si seleccionó *Work On Site*, ingrese el número físico de su cubículo/estación de trabajo para permitir que el técnico de soporte acuda físicamente si es necesario. Si seleccionó *Work From Home*, este campo se deshabilitará automáticamente y se guardará como `WFH`.

Una vez completado el formulario, haga clic en el botón "**Start Chat**".

4.4 Operación del Chat y Categorías

En el panel izquierdo se listan 8 categorías específicas. Seleccione la que corresponda a su problema actual:

- **Performance:** Lentitud de la PC, congelamientos de sistema o demoras al abrir archivos.
- **App Crashing:** Aplicaciones de producción (CRM, marcadores) que se cierran solas.
- **Login / Account:** Problemas de acceso, contraseñas bloqueadas o expiradas.
- **VPN Issues:** Fallas en la conexión remota VPN (FortiClient u otros).
- **Audio / Headset:** Micrófonos desconectados, falta de salida de audio en llamadas.
- **Printer Issues:** Impresoras sin comunicación o cola de impresión detenida.
- **Email / Outlook:** Fallas al recibir correos o perfiles corruptos.
- **Other Issue:** Cualquier otra incidencia no clasificada.

Describa brevemente el problema en el cuadro de texto inferior y presione "Send". El bot propondrá una primera solución. Valide las instrucciones físicas en su equipo y responda con los botones "Sí" (si se arregló) o "No" (si persiste). Repita el proceso hasta un máximo de tres soluciones posibles.

5. Panel de Administración (Admin Dashboard)

5.1 Objetivo del Panel de Administración

Consola técnica restrictiva diseñada para que los ingenieros de soporte y supervisores técnicos de Telecom Networks evalúen las métricas operativas en tiempo real, supervisen la conexión del cliente Puppeteer de WhatsApp, configuren exclusiones de mensajería y realicen pruebas directas de prompting sobre las APIs de IA.

5.2 Niveles de Acceso y Enrutamiento de Roles

El sistema opera bajo un esquema estricto de control de privilegios de acceso. Al finalizar la autenticación con Microsoft MSAL, la función de enrutamiento valida si la dirección de correo corresponde a los administradores de TI autorizados listados en la constante `ALLOWED_ADMIN_EMAILS` de ``script.js``:

```
const ALLOWED_ADMIN_EMAILS = [  
  "rurena@tnoutsourcing.com",  
  "ireyes@tnoutsourcing.com"  
];
```

Si el correo se encuentra en esta lista (o comienza con `ireyes@` o `rurena@`), el sistema despliega el **Gateway / Switch Board Screen** para elegir entre ingresar al Chat Widget como usuario común o abrir la consola técnica **Admin Portal**. Si el correo no cumple con esta restricción, el Gateway se omite silenciosamente y el usuario es enrutado directamente a la interfaz del Chat de soporte de usuario final.

5.3 Funciones del Panel de Administración

A. Dashboard View (Métricas y Tickets)

- **Descripción:** Pantalla principal que muestra el resumen cuantitativo de tickets históricos y la lista cronológica de incidentes creados.
- **Ubicación:** Menú lateral > "Dashboard View".

- **Operación:** Muestra tres indicadores principales: Total Tickets, WA Messages (tickets generados vía WhatsApp), y Chat Tickets (tickets generados vía Web). Debajo muestra los últimos 15 tickets. Al hacer clic sobre cualquier ticket de la lista se despliega un **modal estilo correo electrónico profesional** con los datos completos del agente, ubicación de box y el historial completo del log del chat.
- **Caso de Uso:** Revisar incidencias generadas durante la última hora.
- **Riesgos:** Nulo, ya que es una interfaz de visualización de datos de solo lectura.

B. WhatsApp API View (Monitoreo de Conexión del Bot y Enlace QR)

- **Descripción:** Interfaz gráfica que refleja el estado de la conexión de WhatsApp Web y sirve para re-enlazar el bot mediante el escaneo de un código QR.
- **Ubicación:** Menú lateral > "WhatsApp API".
- **Operación:** Si el bot de WhatsApp está conectado con éxito en el servidor Express, se mostrará el badge en color verde con el estado **Connected**. Si se ha cerrado la sesión, cambiará a color amarillo **Waiting for Scan** y renderizará en tiempo real un código QR dinámico. El administrador debe abrir su aplicación móvil de WhatsApp, ir a "Dispositivos Vinculados" y escanear el QR mostrado.
- **Caso de Uso:** Re-enlazar el dispositivo móvil del bot si Meta solicita volver a vincular la cuenta.
- **Riesgos:** Si se expone el QR a terceros no autorizados, estos podrían secuestrar la sesión del bot. Asegurar que solo personal autorizado de TI visualice esta pantalla.

5. Operación Administrativa e Integraciones

C. AI Integrations View (Consola de Pruebas de Prompts y APIs)

- **Descripción:** Panel para realizar pruebas directas sobre las APIs de inteligencia artificial que consumen el backend de forma rápida.
- **Ubicación:** Menú lateral > "AI Integrations".
- **Operación:** Cuenta con dos interfaces diferenciadas:
 1. **Google Gemini API:** Permite ingresar un prompt técnico y verificar la respuesta devuelta por el modelo `gemini-1.5-flash` externo.
 2. **Ollama (Local AI):** Permite verificar si la red corporativa tiene comunicación directa con el servidor local de Ollama. Permite ingresar el nombre del modelo (por defecto `llama3.2`), la IP/URL de la máquina de IA (por defecto `http://192.168.50.233:11434`) y el prompt de prueba.
- **Caso de Uso:** Verificar si la IP de Ollama cambió o diagnosticar tiempos de respuesta de la IA.
- **Riesgos:** La ejecución de múltiples prompts pesados en la sección de Ollama puede generar latencias en el servidor de IA local si está atendiendo conversaciones de usuarios activos.

5.4 Manual de Gestión de Exclusiones en WhatsApp

Para evitar que el bot responda de forma intrusiva a grupos corporativos importantes (por ejemplo: "IT TN RDOM") o a números privados de gerentes y directores, el backend consulta el archivo local de exclusiones `whatsapp_exclusions.json`. El administrador puede editar este archivo para agregar identificadores de WhatsApp de la siguiente manera:

```
{
  "numbers": [
    "18095551234@c.us",
    "18298889999@c.us"
  ],
  "groups": [
    "120363198765432100@g.us"
  ],
  "ignoreAllGroups": true
}
```

Riesgos de Modificación: Un error de sintaxis JSON (como omitir una coma o un corchete de cierre) provocará que el backend de Express falle al analizar el archivo en el evento `client.on('message')` y, por consiguiente, el bot dejará de responder a todos los chats de WhatsApp o ignorará las exclusiones, respondiendo a grupos corporativos.

5.5 Respaldo y Restauración de la Base de Datos de Tickets

La base de datos local del sistema está almacenada por razones de simplicidad y portabilidad en el archivo `support_tickets.json`. Para respaldar esta información:

1. Acceda por SFTP al servidor de Telecom Networks (IP: `192.168.50.20`) con el usuario `root`.
2. Descargue el archivo `/var/www/html/support_assistant/support_tickets.json`.
3. Para restaurar la información en caso de fallo, detenga el servicio PM2 con `pm2 stop support_assistant`, suba el archivo JSON respaldado al servidor sobrescribiendo el existente, y vuelva a iniciar la aplicación con `pm2 start support_assistant`.

6. Mantenimiento del Sistema

6.1 Mantenimiento Preventivo

Para mantener la estabilidad operativa de la plataforma, el administrador debe ejecutar periódicamente las siguientes acciones de mantenimiento preventivo:

- **Limpieza de almacenamiento local:** Puppeteer genera archivos temporales de navegación en la carpeta oculta `.webjs_cache`. Se recomienda limpiar esta carpeta de forma bimestral si se detecta un aumento inusual en el uso del disco en el servidor.
- **Monitoreo de procesos en producción:** Utilice los comandos del gestor de procesos PM2 para comprobar que el consumo de recursos de la aplicación Express se mantenga estable y verificar la cantidad de reinicios automáticos:

```
pm2 list
pm2 show support_assistant
pm2 monit
```

- **Inspección de logs de la aplicación:** Revise los registros del bot y el servidor Express buscando errores frecuentes de APIs o bloqueos en la conexión WebSocket:

```
pm2 logs support_assistant
```

6.2 Actualización de Código y Despliegue de Cambios

El proyecto incluye un script de despliegue automatizado llamado `deploy.js` que implementa cambios locales en el servidor productivo de forma automática. El flujo recomendado para aplicar actualizaciones es el siguiente:

1. Realice y valide los cambios en un entorno de desarrollo local controlado.
2. Asegúrese de tener configuradas las credenciales de SSH en la constante `config` en `deploy.js` (IP de host: `192.168.50.20`, usuario: `root`, contraseña: `nexus.rdom`).
3. Ejecute el script en su terminal local:

```
node deploy.js
```

4. El script copiará secuencialmente los archivos actualizados hacia la ruta remota `/var/www/html/support_assistant` usando SFTP y se conectará mediante SSH para ejecutar el reinicio del proceso en PM2 de manera automática: `pm2 restart support_assistant`.

6.3 Áreas Críticas que No Deben Modificarse Sin Conocimiento

¡PRECAUCIÓN! ÁREAS CRÍTICAS DE CONFIGURACIÓN

Configuraciones fijas de Azure AD: Los valores de `CLIENT_ID` y `TENANT_ID` en ``script.js`` están enlazados directamente con el registro de la aplicación en el portal de Azure Corporativo. Modificar estos tokens de identificación romperá de inmediato el inicio de sesión de todos los usuarios.

Estrategia de Autenticación de WhatsApp: En ``server.js``, la opción `authStrategy: new LocalAuth()` no debe ser reemplazada. Esta opción es la encargada de mantener la sesión vinculada guardando las cookies del navegador. Modificarla forzará a que el código QR expire en cada arranque.

7. Solución de Problemas (Troubleshooting)

7.1 Matriz de Diagnóstico y Solución de Fallas Comunes

Síntoma	Posible Causa	Procedimiento de Diagnóstico	Solución Preventiva / Correctiva
El bot de WhatsApp no responde a los usuarios y el QR no carga en el panel.	La sesión guardada en Puppeteer se ha corrompido o Meta cerró la sesión remotamente.	Ejecute <code>pm2 logs</code> . Busque errores relacionados con "Protocol Error" o fallas de inicio de Puppeteer.	Detenga PM2 (<code>pm2 stop</code>). Elimine por completo la carpeta oculta <code>.webjs_auth</code> . Inicie el servidor de nuevo y vuelva a escanear el QR.
Ollama devuelve un error de conexión ("Connection Refused").	El servicio Ollama en la IP especificada en <code>.env</code> está apagado o no escucha en la red externa.	Realice un <code>ping</code> <code>192.168.50.233</code> desde el servidor Express. Ejecute un <code>curl</code> al puerto de Ollama.	Asegúrese de que el servidor de Ollama esté encendido y que escuche en todas las interfaces configurando la variable <code>OLLAMA_HOST=0.0.0.0</code> al iniciar Ollama.
El Chat de soporte muestra error al enviar tickets.	Se superó el límite de tickets mensuales gratuitos en la cuenta de EmailJS.	Revise la consola del navegador buscando errores 403 o mensajes devueltos por EmailJS.	Acceda al portal de administración de EmailJS para verificar la cuota o reemplace las credenciales del template/servicio.
Fallo de autenticación ("MSAL redirection mismatch").	La URL de redirección en Azure AD no coincide con la URL desde la que accede el usuario.	Verifique la dirección en la barra del navegador. Debe coincidir exactamente con la configurada (debe ser HTTPS).	Acceda a Azure Portal, navegue a "App Registrations" > Authentication y agregue la URL exacta de redirección utilizada.

Síntoma	Posible Causa	Procedimiento de Diagnóstico	Solución Preventiva / Correctiva
El despliegue con <code>deploy.js</code> falla de inmediato.	El firewall del servidor remoto bloquea el puerto de SSH (22) o las credenciales cambiaron.	Compruebe si puede conectarse de forma manual mediante un cliente SSH convencional como PuTTY.	Actualice las credenciales SSH (contraseña o llave) en el objeto <code>config</code> del script <code>deploy.js</code> .

8. Recuperación del Sistema

8.1 Plan de Recuperación ante Desastres (Falla Crítica)

Si el servidor web remoto que aloja Support ALLama queda completamente fuera de servicio debido a una falla del sistema operativo, corte de energía o pérdida de datos, siga estrictamente la siguiente secuencia de recuperación en frío:

Paso 1: Restauración del Entorno Operativo

1. Provisione un nuevo servidor virtual Linux con Ubuntu Server 22.04 LTS (o superior).
2. Asegúrese de configurar la IP estática interna en la red: `192.168.50.20`.
3. Instale las dependencias de Node.js y Chromium necesarias para que Puppeteer se ejecute sin cabecera gráfica en Linux:

```
sudo apt update
sudo apt install -y nodejs npm chromium-browser
sudo apt install -y libxss1 libasound2 libatk1.0-0 libgtk-3-0 gconf-service libgbm-dev
```

4. Instale de forma global el gestor de procesos PM2:

```
sudo npm install -g pm2
```

Paso 2: Clonación y Despliegue de Código

1. Clone el repositorio del código de la aplicación en la ruta `/var/www/html/support_assistant`.
 2. Instale las dependencias locales ejecutando el comando:
- ```
npm install
```
3. Restaure el archivo `.env` desde su almacenamiento seguro de credenciales corporativas de TI.
  4. Restaure el último respaldo disponible del archivo de base de datos `support_tickets.json` y el de exclusiones `whatsapp_exclusions.json` en la carpeta raíz de la aplicación.

### Paso 3: Inicio de Servicios e Inicialización del Bot

1. Inicie el backend Express utilizando PM2:

```
pm2 start server.js --name support_assistant
```

2. Configure PM2 para que el servidor de soporte se inicie de forma automática con el sistema de la siguiente manera:

```
pm2 startup
pm2 save
```

3. Acceda a la URL de administración <https://supportia.telecomnetworks.org/> , inicie sesión con su cuenta corporativa, abra la sección de WhatsApp del menú lateral y proceda a escanear el código QR recién generado utilizando el teléfono corporativo designado para soporte.
4. Realice una prueba enviando un mensaje de prueba a través de la sección "AI Integrations" para garantizar que la comunicación local con la máquina de Ollama y externa con Gemini funcione sin inconvenientes.

## 9. Protocolos de Seguridad

---

### 9.1 Autenticación de Acceso de Principio Cero (Zero Trust)

---

El sistema elimina la existencia de bases de datos locales de contraseñas. Al delegar la validación al servicio Azure AD/Microsoft Entra ID, la plataforma hereda de forma automática todas las directivas de seguridad corporativas globales:

- **Políticas de MFA:** Si el inquilino corporativo exige autenticación multifactor, el usuario la realizará en la pasarela de Microsoft antes de acceder.
- **Expiración y Bloqueo:** Las cuentas suspendidas por el departamento de recursos humanos o TI perderán el acceso a la plataforma al instante.

### 9.2 Blindaje CORS y Seguridad de API en Express

---

Para evitar ataques de inyección de scripts externos o que otros servidores web no autorizados realicen peticiones hacia la API local de Express, el backend incluye un middleware de CORS restringido a los dominios autorizados de la empresa:

```
const cors = require('cors');
app.use(cors({
 origin: ['https://supportia.telecomnetworks.org', 'http://localhost:3001']
}));
```

## 9.3 Directrices de Manejo Seguro de Secretos e Información Sensible

### ¡REGLA CORPORATIVA DE SEGURIDAD DE DATOS!

**Exclusión en Git (.gitignore):** Las credenciales y claves privadas como el `GEMINI_API_KEY`, contraseñas SMTP del correo electrónico corporativo y secretos de sesión de WhatsApp NUNCA deben subirse a repositorios públicos de código en GitHub o GitLab. El archivo `.gitignore` debe tener definidos explícitamente los archivos `.env` y la carpeta `.webjs_auth/` para evitar filtraciones de seguridad.

**Cifrado en Reposo:** Las copias de respaldo del archivo `.env` deben mantenerse almacenadas y cifradas bajo un gestor de secretos corporativos (ej. Vault o Keepass), nunca en documentos de texto plano expuestos en el servidor.

## 10. Mejoras Futuras y Escalabilidad

---

### 10.1 Conexión con Bases de Conocimiento Estáticas (RAG)

---

Para la fase posterior de desarrollo de Support AILlama, se planea implementar un motor RAG (Retrieval-Augmented Generation). Esto permitirá que, antes de generar respuestas genéricas con Ollama, el sistema realice búsquedas semánticas sobre bases de conocimiento locales (KB) de Telecom Networks (como PDFs de procedimientos técnicos internos). La IA podrá así indicar al usuario la solución exacta correspondiente a los sistemas informáticos específicos de la empresa.

### 10.2 Monitoreo Técnico de Recursos de Servidores Remotos

---

Se propone extender la vista del Panel de Administración web incorporando la visualización directa de la telemetría del servidor en tiempo real (uso de CPU, memoria RAM consumida, y estado del disco duro local de la máquina Express y de la máquina Ollama). Esto agilizará el diagnóstico de caídas del sistema de soporte desde la propia interfaz web.

### 10.3 Conversión a Progressive Web App (PWA)

---

Con el objetivo de facilitar el acceso móvil y el reporte de incidentes en entornos corporativos, se adaptará el frontend para cumplir con los estándares PWA. Esto permitirá a los técnicos instalar la aplicación en sus teléfonos móviles de soporte y ordenadores de escritorio, habilitando las notificaciones push nativas ante la creación de tickets urgentes.

# 11. Anexos Técnicos

---

## 11.1 Ejemplo Completo de Configuración (.env)

---

```
Servidor Express Backend Port
PORT=3001

API Key para Google Gemini Cloud Integration
GEMINI_API_KEY=AIzaSyB3X_ExampleSecretGeminiKey4567

Dirección IP de servidor local de Ollama (con puerto)
OLLAMA_HOST=http://192.168.50.233:11434

Credenciales de Email Corporativo para Reportes Semanales (SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=it-support@telecomnetworks.org
SMTP_PASS=abcdefghijklmnop
```

## 11.2 Comandos Útiles de Mantenimiento y Control

| Comando                                    | Entorno de Ejecución                      | Acción y Descripción                                                    |
|--------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------|
| <code>pm2 restart support_assistant</code> | Servidor Web (Linux SSH)                  | Reinicia el servicio de Express.js y el bot de WhatsApp.                |
| <code>pm2 logs support_assistant</code>    | Servidor Web (Linux SSH)                  | Muestra el registro de eventos y logs técnicos en vivo.                 |
| <code>node deploy.js</code>                | Estación local de desarrollo              | Realiza el despliegue automático del código local al servidor por SFTP. |
| <code>rm -rf .webjs_auth</code>            | Servidor Web (Directorio raíz)            | Elimina el caché de WhatsApp para forzar la recreación del código QR.   |
| <code>ollama run llama3.2</code>           | Servidor de IA Local (PowerShell / Linux) | Inicia el modelo Llama 3.2 localmente en la máquina física de IA.       |